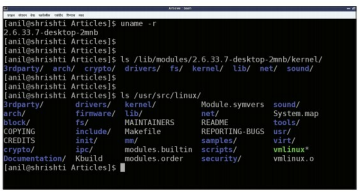

Figure 1: Linux pre-built modules

commonly called modules and built into individual files with a *.ko* (kernel object) extension. Every Linux system has a standard place under the root of the file system (/) for all the pre-built modules. They are organised similar to the kernel source tree structure, under */lib/modules/<kernel_version>/kernel*, where *<kernel_version>* would be the output of the command "*uname -r*" on the system, as shown in Figure 1.

To dynamically load or unload a driver, use these commands, which reside in the */sbin* directory, and must be executed with root privileges:

- *lsmod* – Lists currently loaded modules
- *insmod <module_file>* – Inserts/loads the specified module file
- *modprobe <module>* – Inserts/loads the module, along with any dependencies
- *rmmod <module>* – Removes/unloads the module

Let's look at the FAT filesystem-related drivers as an example. Figure 2 demonstrates this complete process of experimentation. The module files would be *fat.ko*, *vfat.ko*, etc, in the *fat* (and *vfat* for older kernels) directory under */lib/modules/`uname -r`/kernel/fs*. If they are in compressed .gz format, you need to uncompress them with *gunzip*, before you can *insmod* them. The *vfat* module depends on the *fat* module, so *fat.ko* needs to be loaded first. To automatically perform decompression and dependency loading, use *modprobe* instead. Note that you shouldn't specify the *.ko* extension to the module's name, for the *modprobe* command. *rmmod* is used to unload the modules.

Our first Linux driver

Before we write our first driver, let's go over some concepts. A driver never runs by itself. It is similar to a library that is loaded for its functions to be invoked by a running application. Hence, though written in C, it lacks the a *main()* function. Moreover, it will be loaded/linked with the kernel, so it needs to be compiled in a similar way to the kernel, and the header files you can use are only those from the kernel sources, not from the standard */usr/include*.

One interesting fact about the kernel is that it is an object-oriented implementation in C, as we will observe even with our first driver. And it is so profound that

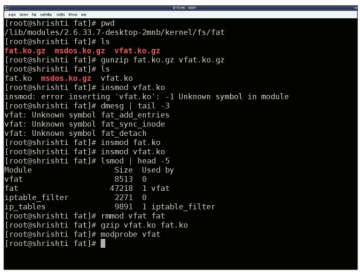

Figure 2: Linux module operations

we would observe the same even with our first driver. Any Linux driver has a constructor and a destructor. The module's constructor is called when the module is successfully loaded into the kernel, and the destructor when *rmmod* succeeds in unloading the module. These two are like normal functions in the driver, except that they are specified as the init and exit functions, respectively, by the macros *module_init()* and *module_exit()*, which are defined in the kernel header *module.h*.

```
/* ofd.c - Our First Driver code */
#include <linux/module.h>
#include <linux/version.h>
#include <linux/kernel.h>

static int __init ofd_init(void) /* Constructor */
{
    printk(KERN_INFO "Namaskar: ofd registered");
    return 0;
}

static void __exit ofd_exit(void) /* Destructor */
{
    printk(KERN_INFO "Alvida: ofd unregistered");
}

module_init(ofd_init);
module_exit(ofd_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Anil Kumar Pugalia <email@sarika-pugs.com>");
MODULE_DESCRIPTION("Our First Driver");
```

Given above is the complete code for our first driver; let's call it *ofd.c*. Note that there is no *stdio.h* (a user-space header); instead, we use the analogous *kernel.h* (a kernel space header). *printk()* is the equivalent of *printf()*. Additionally, *version.h* is included for the module version